BCSE102L
Structured and Object Oriented Programming

C PROGRAMMING LANGUAGE-MODULE-2

By,
Dr. S. Vinila Jinny,
ASP/SCOPE

SYLLABUS

Module:1	C Programming Fundamentals	2 hours
Variables - Reserved words – Data Types – Operators – Operator Precedence - Expressions - Type Conversions - I/O statements - Branching and Looping: if, if-else, nested if, if-else ladder, switch statement, goto statement - Loops: for, while and do…while – break and continue statements.		
Module:2	**Arrays and Functions**	**4 hours**
Arrays: One Dimensional array - Two-Dimensional Array – Strings and its operations. User Defined Functions: Declaration – Definition – call by value and call by reference - Types of Functions - Recursive functions - Storage Classes - Scope, Visibility and Lifetime of Variables.		
Module:3	**Pointers**	**4 hours**
Declaration and Access of Pointer Variables, Pointer arithmetic – Dynamic memory allocation – Pointers and arrays - Pointers and functions.		
Module:4	**Structure and Union**	**2 hours**
Declaration, Initialization, Access of Structure Variables - Arrays of Structure - Arrays within Structure - Structure within Structures - Structures and Functions – Pointers to Structure -		

FUNCTIONS IN C LANGUAGE:-

- A function is a block of code which only runs when it is called.
- we can pass data, known as parameters, into a function.
- Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.

Advantage of function:-

1) Code Resuability
2) Code optimization

SYNTAX TO DECLARE FUNCTION:-

Function Declaration:

```
return_type function_name(data_type parameter);
```

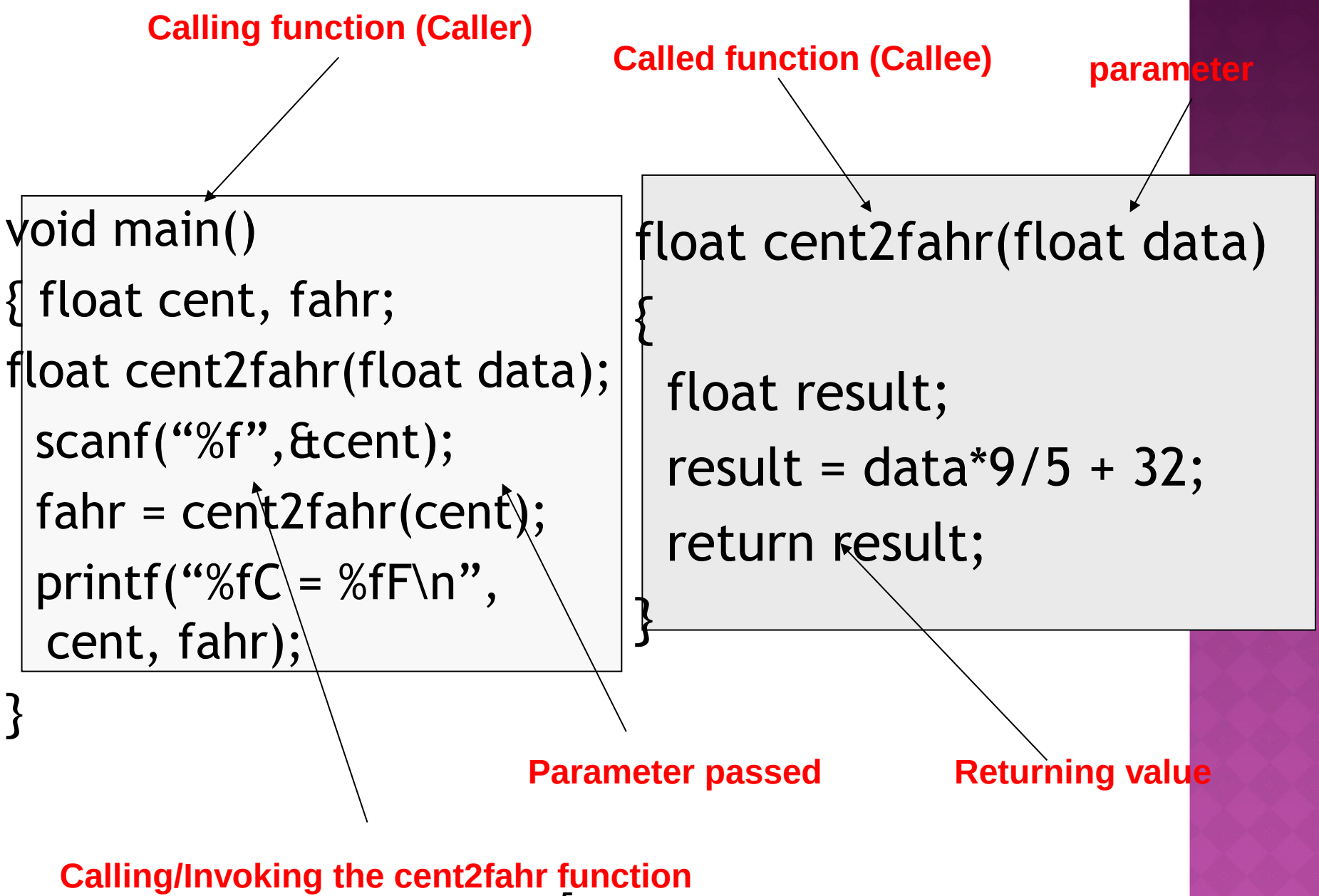
Function Definition

```
return_type function_name(data_type parameter)
{
//code to be executed
}
```

Function Invocation

- ***Syntax to call function:-***

```
variable=function_name(arguments...);
```


Calling function (Caller)
Called function (Callee)
parameter
void main()
{ float cent, fahr;
float cent2fahr(float data);
scanf("%f",&cent);
fahr = cent2fahr(cent);
printf("%fC = %fF\n",
cent, fahr);
}
float cent2fahr(float data)
{
float result;
result = data*9/5 + 32;
return result;
}
Parameter passed
Returning value
Calling/Invoking the cent2fahr function

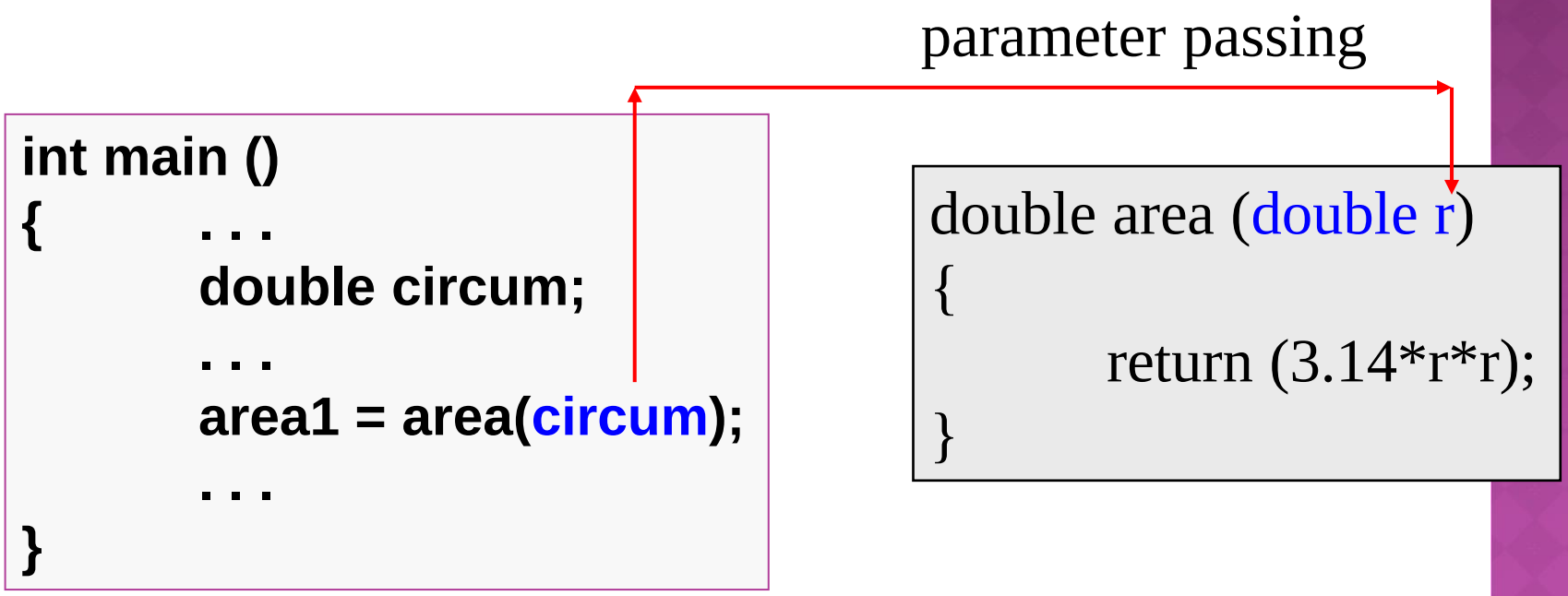
DEFINING A FUNCTION

- A function definition has two parts:
  - The first line, called header
  - The body of the function

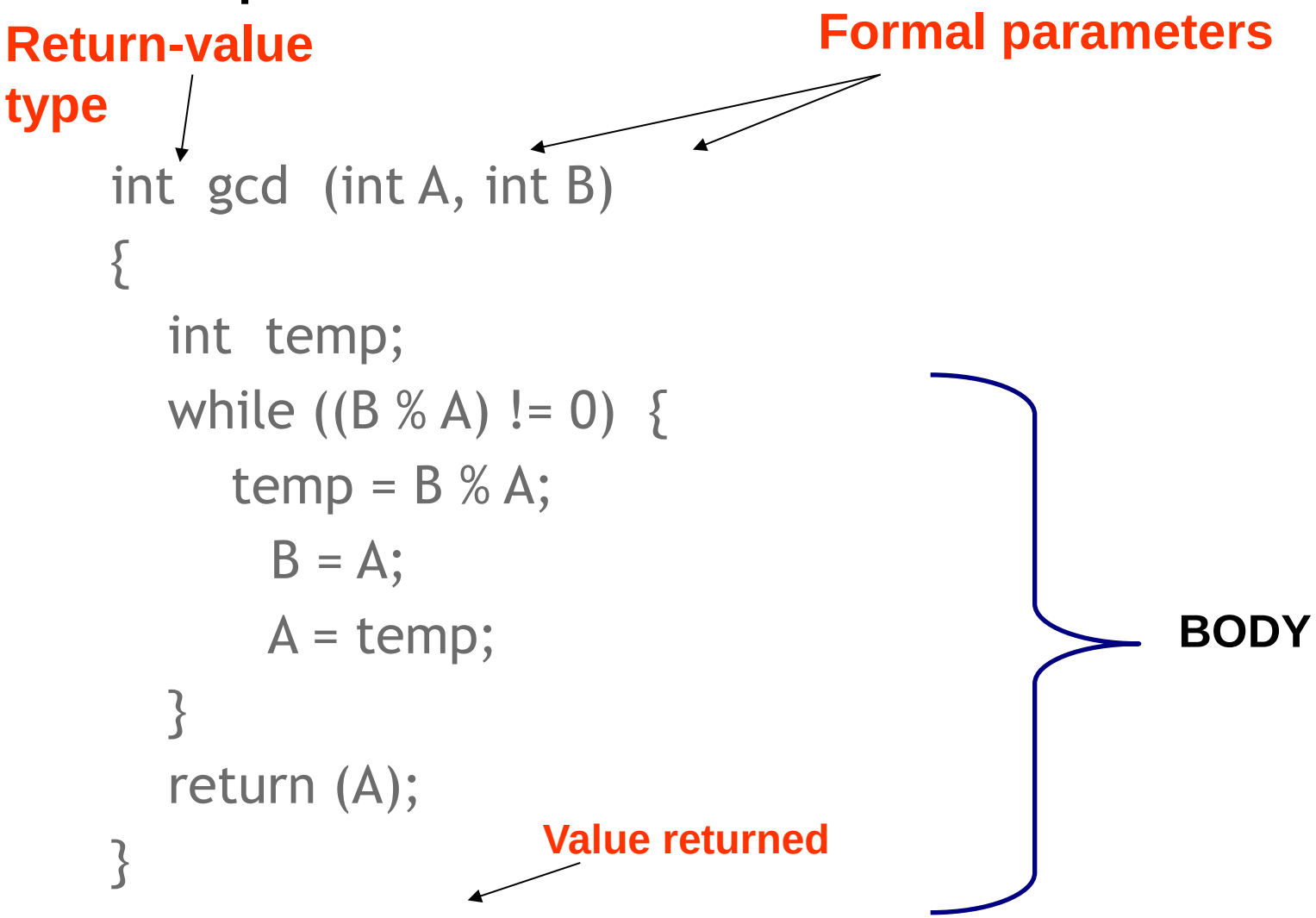
```
return-value-type  function-name  ( parameter-list )
{
    declarations and statements
}
```

PARAMETER PASSING

- When the function is executed, the value of the actual parameter is copied to the formal parameter



Example of function definition



RETURN VALUE

- A function can return a value
  - Using **return** statement
- Like all values in C, a function return value has a type
- The return value can be assigned to a variable in the caller

```
int x, y, z;
scanf("%d%d", &x, &y);
 z = gcd(x,y);
printf("GCD of %d and %d is %d\n", x, y, z);
```

LOCAL VARIABLES

```
/* Find the area of a circle with diameter d */
double circle_area (double d)
{
        double radius, area;
        radius = d/2.0;
        area = 3.14*radius*radius;
        return (area);
}
```

POINTS TO NOTE

- The identifiers used as formal parameters are “local”.
  - Not recognized outside the function
  - Names of formal and actual arguments may differ
- A value-returning function is called by including it in an expression
  - A function with return type T (≠ void) can be used anywhere an expression of type T

SOME MORE POINTS

- A function cannot be defined within another function
  - All function definitions must be disjoint
- Nested function calls are allowed
  - A calls B, B calls C, C calls D, etc.
  - The function called last will be the first to return
- A function can also call itself, either directly or in a cycle
  - A calls B, B calls C, C calls back A.
  - Called recursive call or recursion

EXAMPLE: MAIN CALLS NCR, NCR CALLS FACT

```
int ncr (int n, int r);
int fact (int n);

void main()
{
   int i, m, n, sum=0;
   scanf ("%d %d", &m, &n);
   for (i=1; i<=m; i+=2)
       sum = sum + ncr (n, i);
   printf ("Result: %d \n",
   sum);
}
```

```
int  ncr (int n, int r)
{
return(fact(n) /(fact(r) *fact(n-r)));
}

int  fact (int n)
{
   int  i, temp=1;
   for (i=1; i<=n; i++)
       temp *= i;
   return (temp);
}
```

TYPES OF FUNCTIONS

- Userdefined
  - User writing own functions
- Predefined
  - getch()
  - scanf()

ASSIGNMENT

- Write a C Program to perform Arithmetic Calculations
- 1.Addition
- 2.Subtraction
- 3.Multiplication
- 4.Division

Use While, switch, and function concept.

SAMPLE OUTPUT

```
Artithmetic Calculator
1.ADD
2.SUB
3.MUL
4.DIV
Enter your Choice: 2
SUBTRACTION
Enter two numbers : 5 3
SUB = 2
Need to continue(Y/N):N
```

CALL BY VALUE IN C LANGUAGE:-

- In call by value, value being passed to the function is locally stored by the function parameter in stack memory location.
- If you change the value of function parameter, it is changed for the current function only.
- It will not change the value of variable inside the caller method such as main().

EXAMPLE OF CALL BY VALUE:-

```
#include <stdio.h>
#include <conio.h>
void change(int num) {
    printf("Before adding value inside function num=%d \n",num);
    num=num+100;
    printf("After adding value inside function num=%d \n", num);
}
  int main() {
    int x=100;
    clrscr();
   printf("Before function call x=%d \n", x);
    change(x);//passing value in function
    printf("After function call x=%d \n", x);
   getch();
    return 0;
}
```

OUTPUT WINDOW :-

```
Before function call x=100
Before adding value inside function num=100
After adding value inside function num=200
After function call x=100
```

CALL BY REFERENCE IN C:-

- In call by reference, original value is modified because we pass reference (address).

EXAMPLE OF CALL BY REFERENCE:-

```
#include <stdio.h>
#include <conio.h>
void change(int *num) {
    printf("Before adding value inside function num=%d \n",*num);
    (*num) += 100;
    printf("After adding value inside function num=%d \n", *num);
}

int main() {
    int x=100;
    clrscr();
    printf("Before function call x=%d \n", x);
    change(&x);//passing reference in function
    printf("After function call x=%d \n", x);

    getch();
    return 0;
}
```

OUTPUT WINDOW:-

```
Before function call x=100
Before adding value inside function num=100
After adding value inside function num=200
After function call x=200
```

RECURSION IN C:-

- Recursion is the technique of making a function call itself.
- This technique provides a way to break complicated problems down into simple problems which are easier to solve.
- A function that calls itself, and doesn't perform any task after function call, is know as **tail recursion**. In tail recursion, we generally call the same function with return statement.

➢ ***Syntax:-***

```
recursionfunction(){
  recursionfunction(); //calling self function

}
```

FACTORIAL OF A NUMBER USING RECURSION

```
#include<stdio.h>
long int multiplyNumbers(int n);
int main() {
    int n;
    printf("Enter a positive integer: ");
    scanf("%d",&n);
    printf("Factorial of %d = %ld", n, multiplyNumbers(n));
    return 0;
}
long int multiplyNumbers(int n) {
    if (n>=1)
        return n*multiplyNumbers(n-1);
    else
        return 1;
}
```

TYPE MODIFIERS

- A type modifier alters the meaning of the base type to more precisely fit a specific need.
- signed
- unsigned
- long
- short
- The **int** base type can be modified by **signed**, **short**, **long**, and **unsigned**.
- The **char** type can be modified by **unsigned** and **signed**.

TYPE MODIFIERS

Type	Typical Size in Bits	Minimal Range
char	8	–127 to 127
unsigned char	8	0 to 255
signed char	8	–127 to 127
int	16 or 32	–32,767 to 32,767
unsigned int	16 or 32	0 to 65,535
signed int	16 or 32	Same as **int**
short int	16	–32,767 to 32,767
unsigned short int	16	0 to 65,535
signed short int	16	Same as **short int**
long int	32	–2,147,483,647 to 2,147,483,647
long long int	64	$-(2^{63} - 1)$ to $2^{63} - 1$ (Added by C99)
signed long int	32	Same as **long int**
unsigned long int	32	0 to 4,294,967,295
unsigned long long int	64	$2^{64} - 1$ (Added by C99)
float	32	1E–37 to 1E+37 with six digits of precision
double	64	1E–37 to 1E+37 with ten digits of precision
long double	80	1E–37 to 1E+37 with ten digits of precision

TYPE QUALIFIERS

- C defines type qualifiers that control how variables may be accessed or modified.
- **const** and **volatile**.
- The type qualifiers must precede the type names that they qualify.

CONST

- Variables of type **const** may not be changed by our program.
- A **const** variable can be given an initial value.
- The compiler is free to place variables of this type into read-only memory
- (ROM).
- For example,

```
const int a=10;
```

- creates an integer variable called **a** with an initial value of 10 that your program may not modify.

VOLATILE

- The modifier **volatile** tells the compiler that a variable's value may be changed in ways not explicitly specified by the program.
- can use **const** and **volatile** together

STORAGE CLASSES

- These specifiers tell the compiler how to store the subsequent variable.
- storage class represents the visibility and a location of a variable.
- It tells from what part of code we can access a variable.
- The location where the variable will be stored.
- The initialized value, life time and accessibility of a variable.

TYPES OF STORAGE CLASSES IN C

Storage Classes	Storage Place	Default Value	Scope	Lifetime
auto	RAM	Garbage Value	Local	Within function
extern	RAM	Zero	Global	Till the end of the main program Maybe declared anywhere in the program
static	RAM	Zero	Local	Till the end of the main program, Retains value between multiple functions call
register	Register	Garbage Value	Local	Within the function

AUTO

- Auto stands for automatic storage class. A variable is in auto storage class by default if it is not explicitly specified.
- The scope of an auto variable is limited with the particular block only. Once the control goes out of the block, the access is destroyed. This means only the block in which the auto variable is declared can access it.
- By default, an auto variable contains a garbage value and is local variable.

AUTO-EXAMPLE

```
#include <stdio.h>
int main( )
{
  auto int j = 1;
  {
    auto int j= 2;
    {
      auto int j = 3;
      printf ( " %d ", j);
    }
    printf ( "\t %d ",j);
  }
  printf( "%d\n", j);}
```

Output:
3 2 1

EXTERN

- Extern stands for external storage class. Extern storage class is used when we have global functions or variables which are shared between two or more files.

- Keyword extern is used to declaring a global variable or function in another file to provide the reference of variable or function which have been already defined in the original file.

- The variables defined using an extern keyword are called as global variables. These variables are accessible throughout the program.
- Notice that the extern variable cannot be initialized it has already been defined in the original file.

EXTERN-EXAMPLE

First File: main.c

```
#include <stdio.h>
extern i;
main() {
   printf("value of the external integer is = %d\n", i);
   return 0;}
```

Second File: original.c

```
#include <stdio.h>
i=48;
```

STATIC

- Static local variable is a local variable that retains and stores its value between function calls or block and remains visible only to the function or block in which it is defined.
- static variable has a default initial value zero and is initialized only once in its lifetime.
- Global variables are accessible throughout the file whereas static variables are accessible only to the particular part of a code.
- The lifespan of a static variable is in the entire program code.

STATIC -EXAMPLE

```
#include <stdio.h> /* function declaration */
void next(void);
static int counter = 7; /* global variable */
main() {
 while(counter<10) {
      next();
      counter++;    }
return 0;}
void next( void ) {    /* function definition */
   static int iteration = 13; /* local static variable */
   iteration ++;
   printf("iteration=%d and counter= %d\n", iteration, counter);}
```

Output

```
iteration=14 and counter= 7
iteration=15 and counter= 8
iteration=16 and counter= 9
```

REGISTER

- we can use the register storage class when we want to store local variables within functions or blocks in CPU registers instead of RAM to have quick access to these variables.
- It is similar to the auto storage class. The variable is limited to the particular block.
- The only difference is that the variables declared using register storage class are stored inside CPU registers instead of a memory. Register has faster access than that of the main memory.

EXAMPLE

```
#include <stdio.h> /* function declaration */

main() {

{register int  weight;

int *ptr=&weight ;/*it produces an error when the compilation occurs ,we cannot get a

memory location when dealing with CPU register*/}

}
```

output

```
error: address of register variable 'weight' requested
```

SUMMARY

Storage Class	Declaration	Storage	Default Initial Value	Scope	Lifetime
auto	Inside a function/block	Memory	Unpredictable	Within the function/block	Within the function/block
register	Inside a function/block	CPU Registers	Garbage	Within the function/block	Within the function/block
extern	Outside all functions	Memory	Zero	Entire the file and other files where the variable is declared as extern	program runtime
Static (local)	Inside a function/block	Memory	Zero	Within the function/block	program runtime
Static (global)	Outside all functions	Memory	Zero	Global	program runtime

ARRAY IN C:-

- Array in C language is a *collection* or *group* of elements (data). All the elements of array are *homogeneous*(similar). It has contiguous memory location.

Declaration of array:-

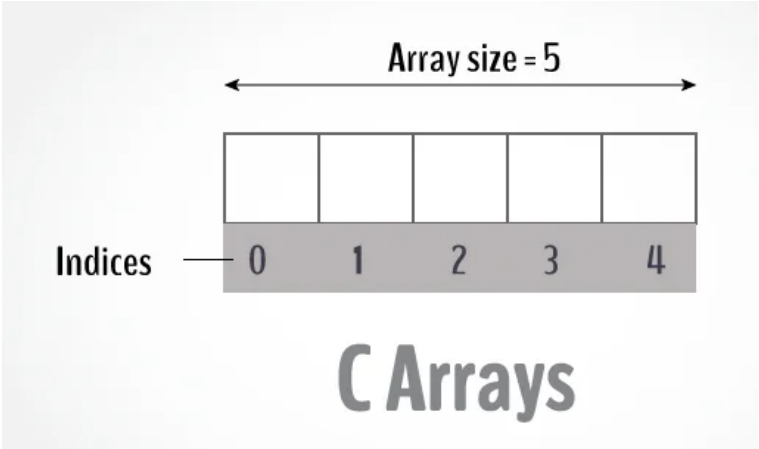
- data_type array_name[array_size];

Eg:-

- **int** marks[7];

Types of array:-

1) 1-D Array
2) 2-D Array



ADVANTAGE OF ARRAY:-

1) Code Optimization
2) Easy to traverse data
3) Easy to sort data
4) Random Access

2-D ARRAY IN C:-

- 2-d Array is represented in the form of rows and columns, also known as matrix. It is also known as *array of arrays* or *list of arrays*.

Declaration of 2-d array:-

- data_type array_name[size1][size2];

INITIALIZATION OF 2-D ARRAY:-

```
int arr[3][4]={{1,2,3,4},{2,3,4,5},{3,4,5,6}};
```

	C1	C2	C3	C4
R1	1	2	3	4
R2	2	3	4	5
R3	3	4	5	6

VARIATIONS OF DECLARATION

- /* Valid declaration*/
- int abc[2][2] = {1, 2, 3 ,4 }
- /* Valid declaration*/
- int abc[][2] = {1, 2, 3 ,4 }
- /* Invalid declaration – you must specify second dimension*/
- int abc[][] = {1, 2, 3 ,4 }
- /* Invalid because of the same reason mentioned above*/
- int abc[2][] = {1, 2, 3 ,4 }

EXAMPLE-1D ARRAY

```
#include <stdio.h>

int main() {

  int values[5];

  printf("Enter 5 integers: ");

  // taking input and storing it in an array

  for(int i = 0; i < 5; ++i) {

     scanf("%d", &values[i]);   }

  printf("Displaying integers: ");

  // printing elements of an array

  for(int i = 0; i < 5; ++i) {

     printf("%d\n", values[i]);   }

  return 0;

}
```

Output:
Enter 5 integers: 1
-3
34
0
3
Displaying integers: 1
-3
34
0
3

EXAMPLE-2D ARRAY

```
#include <stdio.h>
int main()
{
  float a[2][2], b[2][2], result[2][2];
printf("Enter elements of 1st matrix\n");
  for (int i = 0; i < 2; ++i)
    for (int j = 0; j < 2; ++j)
    {
      printf("Enter a%d%d: ", i + 1, j + 1);
      scanf("%f", &a[i][j]);
    }
printf("Enter elements of 2nd matrix\n");
  for (int i = 0; i < 2; ++i)
    for (int j = 0; j < 2; ++j)
    {
      printf("Enter b%d%d: ", i + 1, j + 1);
      scanf("%f", &b[i][j]);
    }
for (int i = 0; i < 2; ++i)
    for (int j = 0; j < 2; ++j)
    {
      result[i][j] = a[i][j] + b[i][j];
    }

printf("\nSum Of Matrix:");

  for (int i = 0; i < 2; ++i)
    for (int j = 0; j < 2; ++j)
    {
      printf("%.1f\t", result[i][j]);

      if (j == i)
        printf("\n");
    }
  return 0;
}
```

OUTPUT

```
Enter elements of 1st matrix
Enter a11: 2;
Enter a12: 0.5;
Enter a21: -1.1;
Enter a22: 2;
Enter elements of 2nd matrix
Enter b11: 0.2;
Enter b12: 0;
Enter b21: 0.23;
Enter b22: 23;

Sum Of Matrix:
2.2     0.5
-0.9    25.0
```

STRINGS

- A string in C is like an array of characters, pursued by a NULL character.
- For example: char c[] = “c string”

Syntax to Define the String in C

- char string_variable_name [array_size];
- char c[] = “c string”

STRING DECLARATION

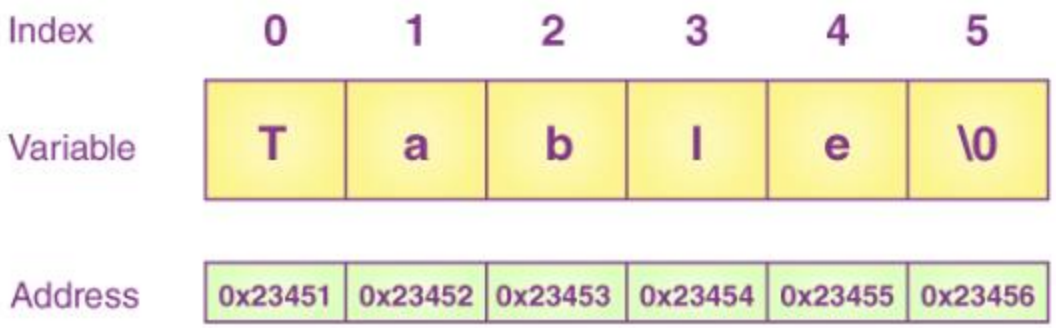
- two ways to declare a string in C language.
  - By char array
  - By string literal

1. By Char Array

char greeting[6] = {‘T’, ‘a’, ‘b’, ‘l’, ‘e’, ‘\0’};

2. By String Literal

- char greeting[] = “Table”;



FUNCTIONS OF STRING

- strcpy(s1, s2); - used to copy string s2 into string s1.
- strcat(s1, s2); - used to concatenate string s2 onto the end of string s1.
- strlen(s1); - It is used to return the length of string.
- strcmp(s1, s2); - It is used to analyse and compare the strings s1 and s2. Returns 0 if s1 and s2 are the same; less than 0 if s1<s2; greater than 0 if s1>s2.

FUNCTIONS OF STRING

- strchr(s1, ch); – It is used to discover the foremost event or occurrence of a specified character within the actual string.
- strstr(s1, s2); – It is used to return a pointer to a character at the first index
- strrev(s); - reverses a given string
- strlwr(s); - Converts string to lowercase
- strupr(s); - Converts string to uppercase

STRING EXAMPLE

```
#include <stdio.h>

int main ()
 {
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
printf("Greeting message: %s\n", greeting );
 return 0;
}
```

Output:

```
Greeting message: Hello
```

```
#include <stdio.h>
#include <string.h>
 void main()
{
char string[25],
reverse_string[25] = {'\0'};
int  i, length = 0, flag = 0;
printf("Enter a string: \n");
gets(string);
for (i = 0; string[i] != '\0'; i++)
    {
        length++;
    }
 for (i = length - 1; i >= 0; i--)
    {
 reverse_string[length - i - 1]
= string[i];
    }
for (i = 0; i < length; i++)
    {
      if (reverse_string[i] == string[i])
            flag = 1;
        else
            {
            flag = 0;
             break;
            }
    }
    if (flag == 1)
printf("%s is a palindrome \n", string);
    else
 printf("%s is not a palindrome \n",
string);
}
```

THANK YOU